Detecting Suspicious Login Activity Using SQL

Objective: Investigate potential security incidents using SQL-based log analysis
Tools: SQL
Scenario:
Suspicious login activity was identified within employee systems, requiring analysis of authentication logs.
Investigation:

- Queried failed login attempts occurring after business hours
- Retrieved login attempts across specific dates tied to suspicious activity
- Identified login attempts from unexpected geographic locations
- Filtered employee records by department for targeted system updates

Key Queries Used:

```
SELECT *
FROM log_in_attempts
WHERE login_time > '18:00' AND success = FALSE;
```

Findings:

- Presence of failed login attempts after hours
- Suspicious login patterns across multiple dates
- Potential unauthorized access from outside expected regions

Response Actions:

- Flagged accounts for further investigation
- Recommended monitoring for brute-force attempts
- Suggested tightening access controls

Outcome:

- Improved visibility into authentication threats
- Demonstrated ability to detect anomalies using data analysis

Apply filters to SQL queries

Project description

I am a security professional at a large organization. Part of my job is to investigate security issues to help keep the system secure. I recently discovered some potential security issues that involve login attempts and employee machines. My task is to examine the organization's data in their employees and log_in_attempts tables. I will use SQL filters to retrieve records from different datasets and investigate the potential security issues.

Retrieve after hours failed login attempts

I recently discovered a potential security incident that occurred after business hours. To investigate this, I need to query the log_in_attempts table and review after hours login activity. I will use filters in SQL to create a query that identifies all failed login attempts that occurred after 18:00. (The time of the login attempt is found in the login_time column. The success column contains a value of 0 when a login attempt failed; you can use either a value of 0 or FALSE in your query to identify failed login attempts.)

```
MariaDB [organization]> SELECT *
    ->
    -> FROM log_in_attempts
    ->
    -> WHERE login_time > '18:00' AND success = FALSE;
+----------+----------+------------+------------+---------+-----------------+---------+
| event_id | username | login_date | login_time | country | ip_address      | success |
+----------+----------+------------+------------+---------+-----------------+---------+
|        2 | apatel   | 2022-05-10 | 20:27:27   | CAN     | 192.168.205.12  |       0 |
|       18 | pwashing | 2022-05-11 | 19:28:50   | US      | 192.168.66.142  |       0 |
|       20 | tshah    | 2022-05-12 | 18:56:36   | MEXICO  | 192.168.109.50  |       0 |
|       28 | aestrada | 2022-05-09 | 19:28:12   | MEXICO  | 192.168.27.57   |       0 |
|       34 | drosas   | 2022-05-11 | 21:02:04   | US      | 192.168.45.93   |       0 |
|       42 | cgriffin | 2022-05-09 | 23:04:05   | US      | 192.168.4.157   |       0 |
|       52 | cjackson | 2022-05-10 | 22:07:07   | CAN     | 192.168.58.57   |       0 |
|       69 | wjaffrey | 2022-05-11 | 19:55:15   | USA     | 192.168.100.17  |       0 |
|       82 | abernard | 2022-05-12 | 23:38:46   | MEX     | 192.168.234.49  |       0 |
|       87 | apatel   | 2022-05-08 | 22:38:31   | CANADA  | 192.168.132.153 |       0 |
|       96 | ivelasco | 2022-05-09 | 22:36:36   | CAN     | 192.168.84.194  |       0 |
|      104 | asundara | 2022-05-11 | 18:38:07   | US      | 192.168.96.200  |       0 |
|      107 | bisles   | 2022-05-12 | 20:25:57   | USA     | 192.168.116.187 |       0 |
|      111 | aestrada | 2022-05-10 | 22:00:26   | MEXICO  | 192.168.76.27   |       0 |
|      127 | abellmas | 2022-05-09 | 21:20:51   | CANADA  | 192.168.70.122  |       0 |
|      131 | bisles   | 2022-05-09 | 20:03:55   | US      | 192.168.113.171 |       0 |
|      155 | cgriffin | 2022-05-12 | 22:18:42   | USA     | 192.168.236.176 |       0 |
|      160 | jclark   | 2022-05-10 | 20:49:00   | CANADA  | 192.168.214.49  |       0 |
|      199 | yappiah  | 2022-05-11 | 19:34:48   | MEXICO  | 192.168.44.232  |       0 |
+----------+----------+------------+------------+---------+-----------------+---------+
19 rows in set (0.005 sec)
```

```
SELECT *
FROM log_in_attempts
```

```
WHERE login_time > '18:00' AND success = FALSE;
```

This will query the log_in attempts table and return all of the columns. When I used `*` in the query, it means that I am selecting all of the columns in the table. I am using the `WHERE` clause to filter only those rows where the login time is after 6 PM and where the user did not login successfully.

Retrieve login attempts on specific dates

My team is investigating a suspicious event that occurred on `'2022-05-09'`. I want to retrieve all login attempts that occurred on this day and the day before (`'2022-05-08'`)

```
MariaDB [organization]> SELECT *
    ->
    -> FROM log_in_attempts
    ->
    -> WHERE login_date = '2022-05-08' OR login_date = '2022-05-09';
+----------+----------+------------+------------+---------+-----------------+---------+
| event_id | username | login_date | login_time | country | ip_address      | success |
+----------+----------+------------+------------+---------+-----------------+---------+
|        1 | jrafael  | 2022-05-09 | 04:56:27   | CAN     | 192.168.243.140 |       1 |
|        3 | dkot     | 2022-05-09 | 06:47:41   | USA     | 192.168.151.162 |       1 |
|        4 | dkot     | 2022-05-08 | 02:00:39   | USA     | 192.168.178.71  |       0 |
|        8 | bisles   | 2022-05-08 | 01:30:17   | US      | 192.168.119.173 |       0 |
|       12 | dkot     | 2022-05-08 | 09:11:34   | USA     | 192.168.100.158 |       1 |
|       15 | lyamamot | 2022-05-09 | 17:17:26   | USA     | 192.168.183.51  |       0 |
|       24 | arusso   | 2022-05-09 | 06:49:39   | MEXICO  | 192.168.171.192 |       1 |
|       25 | sbaelish | 2022-05-09 | 07:04:02   | US      | 192.168.33.137  |       1 |
|       26 | apatel   | 2022-05-08 | 17:27:00   | CANADA  | 192.168.123.105 |       1 |
|       28 | aestrada | 2022-05-09 | 19:28:12   | MEXICO  | 192.168.27.57   |       0 |
|       30 | yappiah  | 2022-05-09 | 03:22:22   | MEX     | 192.168.124.48  |       1 |
```

```
SELECT *
FROM log_in_attempts
WHERE login_date = '2022-05-08' OR login_date = '2022-05-09';
```

This query will select all columns from the log_in_attempts table. I am only selecting the rows for the login dates on 2022-05-08 and 2022-05-09.

Retrieve login attempts outside of Mexico

I am investigating logins that did not originate in Mexico, and I need to find this information.

```
MariaDB [organization]> SELECT *
    ->
    -> FROM log_in_attempts
    ->
    -> WHERE NOT country LIKE 'MEX%';
+----------+----------+------------+------------+---------+----------------+---------+
| event_id | username | login_date | login_time | country | ip_address     | success |
+----------+----------+------------+------------+---------+----------------+---------+
|        1 | jrafael  | 2022-05-09 | 04:56:27   | CAN     | 192.168.243.140 |       1 |
|        2 | apatel   | 2022-05-10 | 20:27:27   | CAN     | 192.168.205.12  |       0 |
|        3 | dkot     | 2022-05-09 | 06:47:41   | USA     | 192.168.151.162 |       1 |
|        4 | dkot     | 2022-05-08 | 02:00:39   | USA     | 192.168.178.71  |       0 |
|        5 | jrafael  | 2022-05-11 | 03:05:59   | CANADA  | 192.168.86.232  |       0 |
|        7 | eraab    | 2022-05-11 | 01:45:14   | CAN     | 192.168.170.243 |       1 |
|        8 | bisles   | 2022-05-08 | 01:30:17   | US      | 192.168.119.173 |       0 |
|       10 | jrafael  | 2022-05-12 | 09:33:19   | CANADA  | 192.168.228.221 |       0 |
|       11 | sgilmore | 2022-05-11 | 10:16:29   | CANADA  | 192.168.140.81  |       0 |
```

```
SELECT *
FROM log_in_attempts
WHERE NOT country LIKE 'MEX%';
```

This will return all the columns form the log_in_attempts table. The `WHERE` clause will filter onm the rows and return only those where the country is not Mexico. I used `LIKE` because some of the items use the word Mexico and so use the country code like MEX, so using the `%` wildcard will filter the table properly.

Retrieve employees in Marketing

I am updating employee machines, and I need to obtain the information about employees in the `'Marketing'` department who are located in all offices in the East building (such as `'East-170'` or `'East-320'`).

```
MariaDB [organization]> SELECT *
    ->
    -> FROM employees WHERE department LIKE 'Marketing' AND office LIKE 'East%';
+-------------+--------------+----------+------------+----------+
| employee_id | device_id    | username | department | office   |
+-------------+--------------+----------+------------+----------+
|        1000 | a320b137c219 | elarson  | Marketing  | East-170 |
|        1052 | a192b174c940 | jdarosa  | Marketing  | East-195 |
|        1075 | x573y883z772 | fbautist | Marketing  | East-267 |
|        1088 | k8651965m233 | rgosh    | Marketing  | East-157 |
|        1103 | NULL         | randerss | Marketing  | East-460 |
|        1156 | a184b775c707 | dellery  | Marketing  | East-417 |
|        1163 | h679i515j339 | cwilliam | Marketing  | East-216 |
+-------------+--------------+----------+------------+----------+
7 rows in set (0.002 sec)
```

```
SELECT *
FROM employees
WHERE department LIKE 'Marketing' AND office LIKE 'East%';
```

This query will return all of the columns in the employees table but only those rows from the Marketing department and East offices. The EAST% is used because it is easier to use the `%` wildcard after East to select all of the rows with East at the beginning of the office name than listing all of the office names individualy in the query.

Retrieve employees in Finance or Sales

Now, I need to perform a different update to the computers of all employees in the Finance or the Sales department, and I need to locate information on these employees.

```
MariaDB [organization]> SELECT *   FROM employees  WHERE department LIKE 'Finance' OR department LIKE 'Sa
les';
+-------------+--------------+----------+------------+-------------+
| employee_id | device_id    | username | department | office      |
+-------------+--------------+----------+------------+-------------+
|        1003 | d394e816f943 | sgilmore | Finance    | South-153   |
|        1007 | h174i497j413 | wjaffrey | Finance    | North-406   |
|        1008 | i858j583k571 | abernard | Finance    | South-170   |
|        1009 | NULL         | lrodriqu | Sales      | South-134   |
|        1010 | k2421212m542 | jlansky  | Finance    | South-109   |
|        1011 | l748m120n401 | drosas   | Sales      | South-292   |
|        1015 | p611q262r945 | jsoto    | Finance    | North-271   |
|        1017 | r550s824t230 | jclark   | Finance    | North-188   |
|        1018 | s310t540u653 | abellmas | Finance    | North-403   |
|        1022 | w237x430y567 | arusso   | Finance    | West-465    |
```

```
SELECT *
FROM employees
WHERE department LIKE 'Finance' OR department LIKE 'Sales';
```

This will select all the columns from the employee table. The `WHERE` clause is used to filter out only the rows that have a department of Finance or Sales. This is shown in the filter. The `OR` clause is used here because we wanted to see either Finance or Sales department.

Retrieve all employees not in IT

I need to make one more update. This update was already made to employee computers in the Information Technology department. The team needs information about employees who are not in that department.

```
MariaDB [organization]> SELECT * FROM employees WHERE NOT department LIKE 'Information Technology';
+-------------+--------------+----------+-----------------+-------------+
| employee_id | device_id    | username | department      | office      |
+-------------+--------------+----------+-----------------+-------------+
|        1000 | a320b137c219 | elarson  | Marketing       | East-170    |
|        1001 | b239c825d303 | bmoreno  | Marketing       | Central-276 |
|        1002 | c116d593e558 | tshah    | Human Resources | North-434   |
|        1003 | d394e816f943 | sgilmore | Finance         | South-153   |
|        1004 | e218f877g788 | eraab    | Human Resources | South-127   |
|        1005 | f551g340h864 | gesparza | Human Resources | South-366   |
|        1007 | h174i497j413 | wjaffrey | Finance         | North-406   |
|        1008 | i858j583k571 | abernard | Finance         | South-170   |
|        1009 | NULL         | lrodriqu | Sales           | South-134   |
|        1010 | k2421212m542 | jlansky  | Finance         | South-109   |
|        1011 | l748m120n401 | drosas   | Sales           | South-292   |
|        1015 | p611q262r945 | jsoto    | Finance         | North-271   |
|        1016 | q793r736s288 | sbaelish | Human Resources | North-229   |
```

```
SELECT *
FROM employees
WHERE NOT department LIKE 'Information Technology';
```

This will return all of the columns in the employees table but the `WHERE` filter is used to select only the employees that do not belong to IT. It is simpler and easier to use the `NOT` in this case, than to list all the departments that I will need to see.

Summary

SQL queries where used to get information about logins and employees. The `AND`, `OR`, and `NOT` operators were used in the `WHERE` clause to filter out certain items. The `%` wildcard was used with `LIKE` to filter further on certain strings.